

146. LRU 缓存

LRU 缓存

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Medium (54.40%)	3478	0	-	-	0

- Tags
- Companies

请你设计并实现一个满足 LRU (最近最少使用) 缓存 约束的数据结构。

实现 LRUCache 类:

- LRUCache(int capacity) 以 正整数 作为容量 capacity 初始化 LRU 缓存
- int get(int key) 如果关键字 key 存在于缓存中，则返回关键字的值，否则返回 -1。
- void put(int key, int value) 如果关键字 key 已经存在，则变更其数据值 value；如果不存在，则向缓存中插入该组 key-value。如果插入操作导致关键字数量超过 capacity，则应该 逐出 最久未使用的关键字。

函数 get 和 put 必须以 O(1) 的平均时间复杂度运行。

示例:

输入

```
["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]  
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]
```

输出

```
[null, null, null, 1, null, -1, null, -1, 3, 4]
```

解释

```
LRUCache lRUCache = new LRUCache(2);  
lRUCache.put(1, 1); // 缓存是 {1=1}  
lRUCache.put(2, 2); // 缓存是 {1=1, 2=2}  
lRUCache.get(1);    // 返回 1  
lRUCache.put(3, 3); // 该操作会使得关键字 2 作废，缓存是 {1=1, 3=3}  
lRUCache.get(2);    // 返回 -1 (未找到)  
lRUCache.put(4, 4); // 该操作会使得关键字 1 作废，缓存是 {4=4, 3=3}  
lRUCache.get(1);    // 返回 -1 (未找到)  
lRUCache.get(3);    // 返回 3  
lRUCache.get(4);    // 返回 4
```

提示:

- 1 <= capacity <= 3000

- $0 \leq \text{key} \leq 10000$
- $0 \leq \text{value} \leq 105$
- 最多调用 $2 * 105$ 次 `get` 和 `put`

Discussion | Solution

解决方法

LeetCode 146: LRU 缓存（详细解析）

1. 问题理解

目标：设计并实现一个 LRU (Least Recently Used - 最近最少使用) 缓存机制。它应该支持以下操作：

- `LRUCache(int capacity)`: 初始化缓存，给定一个正整数作为容量 `capacity`。
- `int get(int key)`: 获取键 `key` 对应的值。如果键存在，则返回对应的值，并标记该键为最近使用；否则，返回 -1。
- `void put(int key, int value)`: 插入或更新键值对 (`key`, `value`)。
 - 如果键 `key` 已存在，则更新其对应的值，并标记该键为最近使用。
 - 如果键 `key` 不存在，则插入该键值对。
 - 如果插入新键值对后缓存容量超出了 `capacity`，则需要淘汰最久未使用的键值对。

核心要求：`get` 和 `put` 操作的时间复杂度都需要是 $O(1)$ 。

关键点：

- **LRU 策略**：当缓存满时，需要移除最长时间没有被访问（`get` 或 `put`）的数据。
- **$O(1)$ 时间复杂度**：这暗示了我们需要高效的数据结构来支持快速查找、插入、删除和维护访问顺序。

2. 思路分析：选择合适的数据结构

为了同时满足 $O(1)$ 的查找和 $O(1)$ 的插入/删除/顺序维护，单一的数据结构很难做到。我们需要组合使用：

1. 快速查找 ($O(1)$): 哈希表 (HashMap) 是理想选择。我们可以用 `key` 作为哈希表的键，值可以是指向缓存数据的引用。
2. 维护访问顺序 & 快速插入/删除 ($O(1)$): 双向链表 (Doubly Linked List) 非常适合。
 - 我们可以将最近使用的元素放在链表头部。
 - 当缓存满时，最久未使用的元素自然就在链表尾部，可以 $O(1)$ 删除。
 - 访问一个元素（`get` 或 `put` 已存在的 `key`）时，可以将其从链表当前位置 $O(1)$ 删除，并 $O(1)$ 插入到链表头部。

- 插入新元素时， $O(1)$ 插入到链表头部。

组合方案：

- 使用 `HashMap<Integer, DLinkedListNode>`：Key 是缓存的键，Value 是指向双向链表中对应节点的引用。
- 使用自定义的双向链表 `DLinkedListNode`：每个节点存储 `key`, `value`, `prev` 指针, `next` 指针。
- 链表顺序约定：链表头部节点表示最近使用的，链表尾部节点表示最久未使用的。
- 伪头尾节点：使用 `head` 和 `tail` 两个伪节点（dummy nodes）可以极大地简化链表插入和删除的边界条件处理。

3. 数据结构设计

// 定义双向链表节点

```
private static class DLinkedListNode {
    int key;
    int value;
    DLinkedListNode prev;
    DLinkedListNode next;

    public DLinkedListNode() {} // 用于伪节点
    public DLinkedListNode(int key, int value) {
        this.key = key;
        this.value = value;
    }
}
```

// LRUCache 类的成员变量

```
private final Map<Integer, DLinkedListNode> cache; // 哈希表
private final DLinkedListNode head; // 伪头节点
private final DLinkedListNode tail; // 伪尾节点
private final int capacity; // 容量
private int size; // 当前大小
```

初始化 (`LRUCache(int capacity)`):

1. 设置 `capacity` 和 `size = 0`。
2. 初始化 `cache = new HashMap<>()`。
3. 创建伪头节点 `head` 和伪尾节点 `tail`。
4. 将它们连接起来: `head.next = tail; tail.prev = head;`

4. 核心操作实现

辅助方法

为了代码清晰，先定义几个链表操作的辅助方法：

```

// 将节点添加到链表头部（伪头节点之后）
private void addToHead(DLinkedNode node) {
    node.prev = head;
    node.next = head.next;
    head.next.prev = node;
    head.next = node;
}

// 从链表中移除指定节点
private void removeNode(DLinkedNode node) {
    node.prev.next = node.next;
    node.next.prev = node.prev;
}

// 将指定节点移动到链表头部
private void moveToHead(DLinkedNode node) {
    removeNode(node); // 先移除
    addToHead(node);  // 再添加到头部
}

// 移除链表尾部节点（最久未使用，伪尾节点之前）并返回
private DLinkedNode removeTail() {
    DLinkedNode res = tail.prev; // 获取实际的尾节点
    removeNode(res);
    return res;
}

```

`get(int key)` 实现

1. 通过 `cache.get(key)` 查找节点 `node`。
2. 如果 `node == null`，表示键不存在，返回 -1。
3. 如果 `node != null`，表示键存在：
 - 调用 `moveToHead(node)` 将该节点移动到链表头部（标记为最近使用）。
 - 返回 `node.value`。

`put(int key, int value)` 实现

1. 通过 `cache.get(key)` 查找节点 `node`。
2. 如果 `node != null`（键已存在）：
 - 更新节点的值：`node.value = value`。
 - 调用 `moveToHead(node)` 将该节点移动到链表头部。
3. 如果 `node == null`（键不存在）：
 - 创建新节点：`DLinkedNode newNode = new DLinkedNode(key, value)`。
 - 将新节点添加到哈希表：`cache.put(key, newNode)`。
 - 将新节点添加到链表头部：`addToHead(newNode)`。

- 增加缓存大小: `size++`。
- 检查容量: 如果 `size > capacity`:
 - 移除链表尾部节点 (最久未使用): `DLinkedListNode tailNode = removeTail()`。
 - 从哈希表中移除对应条目: `cache.remove(tailNode.key)`。
 - 减少缓存大小: `size--`。

5. 完整代码实现

```
import java.util.HashMap;
import java.util.Map;

/*
 * @lc app=leetcode.cn id=146 lang=java
 * @lcpr version=30204
 *
 * [146] LRU 缓存
 */

// @lcpr-template-start

// @lcpr-template-end
// @lc code=start
class LRUCache {

    // 定义双向链表节点
    private static class DLinkedListNode {
        int key;
        int value;
        DLinkedListNode prev;
        DLinkedListNode next;

        public DLinkedListNode() {} // 用于伪节点
        public DLinkedListNode(int key, int value) {
            this.key = key;
            this.value = value;
        }
    }

    private final Map<Integer, DLinkedListNode> cache; // 哈希表, 用于 O(1) 查找节点
    private final DLinkedListNode head; // 伪头节点
    private final DLinkedListNode tail; // 伪尾节点
    private final int capacity; // 缓存容量
    private int size; // 当前缓存大小

    public LRUCache(int capacity) {
```

```

        this.capacity = capacity;
        this.size = 0;
        this.cache = new HashMap<>();
        // 初始化伪头尾节点，并将它们连接起来
        this.head = new DLinkedNode();
        this.tail = new DLinkedNode();
        head.next = tail;
        tail.prev = head;
    }

    /**
     * 获取 key 对应的 value。
     * 如果 key 存在，将其移动到链表头部（表示最近使用），并返回值。
     * 如果 key 不存在，返回 -1。
     */
    public int get(int key) {
        DLinkedNode node = cache.get(key);
        if (node == null) {
            // key 不存在
            return -1;
        }
        // key 存在，将其移动到头部（标记为最近使用）
        moveToHead(node);
        return node.value;
    }

    /**
     * 插入或更新 key-value 对。
     * 如果 key 已存在，更新 value，并将其移动到链表头部。
     * 如果 key 不存在：
     *   - 创建新节点，插入到链表头部和哈希表中。
     *   - 如果缓存大小超过容量，则移除链表尾部节点（最久未使用）。
     */
    public void put(int key, int value) {
        DLinkedNode node = cache.get(key);

        if (node == null) {
            // key 不存在，创建新节点
            DLinkedNode newNode = new DLinkedNode(key, value);
            // 添加到哈希表
            cache.put(key, newNode);
            // 添加到链表头部
            addToHead(newNode);
            // 增加大小
            size++;
            // 检查是否超出容量

```

```

        if (size > capacity) {
            // 移除链表尾部节点（最久未使用）
            DLinkedListNode tailNode = removeTail();
            // 从哈希表中移除对应的 key
            cache.remove(tailNode.key);
            // 减少大小
            size--;
        }
    } else {
        // key 已存在，更新 value
        node.value = value;
        // 将节点移动到头部（标记为最近使用）
        moveToHead(node);
    }
}

// --- 辅助方法 ---

/**
 * 将节点添加到链表头部（伪头节点之后）
 */
private void addToHead(DLinkedListNode node) {
    node.prev = head;
    node.next = head.next;
    head.next.prev = node;
    head.next = node;
}

/**
 * 从链表中移除指定节点
 */
private void removeNode(DLinkedListNode node) {
    node.prev.next = node.next;
    node.next.prev = node.prev;
}

/**
 * 将指定节点移动到链表头部
 */
private void moveToHead(DLinkedListNode node) {
    // 先从当前位置移除
    removeNode(node);
    // 再添加到头部
    addToHead(node);
}

```

```

/**
 * 移除链表尾部节点（伪尾节点之前，即最久未使用的节点）
 * @return 被移除的节点
 */
private DLinkedListNode removeTail() {
    DLinkedListNode res = tail.prev;
    removeNode(res);
    return res;
}
}

/**
 * Your LRUCache object will be instantiated and called as such:
 * LRUCache obj = new LRUCache(capacity);
 * int param_1 = obj.get(key);
 * obj.put(key,value);
 */
// @lc code=end

```

6. 复杂度分析

- 时间复杂度：
 - `get(key)`: $O(1)$ 。哈希表查找是 $O(1)$ ，链表移动操作（`removeNode`, `addToHead`）也是 $O(1)$ 。
 - `put(key, value)`: $O(1)$ 。哈希表查找/插入是 $O(1)$ ，链表添加/移动/删除尾部节点都是 $O(1)$ 。
- 空间复杂度： $O(\text{capacity})$ 。哈希表和双向链表最多存储 `capacity` 个键值和节点。

7. 如何在面试中讲解

1. 明确需求和约束：
 - “我们需要实现一个 LRU 缓存，支持 `get` 和 `put` 操作，并且这两个操作的时间复杂度必须是 $O(1)$ 。核心在于淘汰最久未使用的数据。”
2. 分析 **$O(1)$** 的挑战：
 - “ $O(1)$ 的查找需要哈希表。”
 - “ $O(1)$ 的插入、删除和维护顺序（找到最久未使用的）需要链表，特别是双向链表，因为它允许 $O(1)$ 删除任意节点。”
3. 提出核心数据结构：
 - “因此，最佳方案是结合使用哈希表和双向链表。”
 - “哈希表的 Key 是缓存键，Value 是指向双向链表中对应节点的引用。”
 - “双向链表按照访问顺序存储节点，链表头表示最近使用，链表尾表示最久未使用。”
 - “使用伪头尾节点可以简化链表操作。”
4. 讲解 `get` 操作流程：
 - “通过哈希表查找节点。”

- “如果找到，将节点从链表当前位置移动到头部，并返回值。”
 - “如果找不到，返回 -1。”
5. 讲解 **put** 操作流程：
- “通过哈希表查找节点。”
 - “如果找到，更新值，并将节点移动到链表头部。”
 - “如果找不到，创建新节点，添加到哈希表和链表头部。”
 - “然后检查容量，如果超限，则移除链表尾部节点，并从哈希表中删除对应项。”
6. 强调辅助函数的作用：
- “为了代码清晰，可以封装链表操作（如 `addToHead`, `removeNode`, `moveToHead`, `removeTail`）为辅助方法。”
7. 总结复杂度：
- “这种哈希表 + 双向链表的实现方式，可以保证 `get` 和 `put` 操作的平均时间复杂度都是 $O(1)$ ，空间复杂度是 $O(\text{capacity})$ 。”
- 找到具有 2 个许可证类型的类似代码